

Lecture 8: Dynamic Programming Algorithms

*Instructor: Sanjukta Roy**Scribe: Farhin and Anvesh*

1 Dynamic Programming Solutions

Problem 1 (Knapsack Problem). *Given:*

- n items: $1, 2, 3, \dots, n$
- n weights: $w_1, w_2, \dots, w_n$
- n costs: $c_1, c_2, \dots, c_n$
- A budget B

Find a subset $S \subseteq [n]$ such that $\sum_{j \in S} c_j$ is maximum and $\sum_{j \in S} w_j \leq B$.

Solution. First, let's try a simpler definition. Let $\text{OPT}(i)$ be the value we get when item i is in the bag, considering only items 1 to i (not 1 to n).

Then:

$$\text{OPT}(i) = c_i + \text{OPT}(i - 1)$$

However, this doesn't account for the weight constraint. Let's refine our definition to include the budget parameter.

Let $\text{OPT}(i, B_i)$ be the value we get when item i is in the bag, considering only items 1 to i , with the constraint that the total weight of the bag must be $\leq B_i$, where B_i ranges from w_i to B .

Note that $\text{OPT}(i, B_i)$ mandates that item i is definitely in the bag. Therefore, the final solution is:

$$\text{Final Solution} = \max_i \{\text{OPT}(i, B_i)\}$$

where

$$\text{OPT}(i, B_i) = c_i + \max_{j < i} \{\text{OPT}(j, B_i - w_i)\}$$

with base case $\text{OPT}(0, 0) = 0$, for $0 \leq i \leq n$ and $0 \leq B_i \leq B$.

Example:

- $c_i = 5, 2, 4, 8, 1$
- $w_i = 10, 5, 2, 2, 3$
- $B = 17$

Then:

$$\text{OPT}(3, 10) = 4 + \max\{\text{OPT}(2, 8), \text{OPT}(1, 8)\}$$

□

Proof of Correctness. We need to prove the recurrence relation:

$$\text{OPT}(i, B_i) = c_i + \max_{j < i} \{\text{OPT}(j, B_i - w_i)\}$$

We prove both inequalities:

1. $\text{OPT}(i, B_i) \geq c_i + \max_{j < i} \{\text{OPT}(j, B_i - w_i)\}$
2. $\text{OPT}(i, B_i) \leq c_i + \max_{j < i} \{\text{OPT}(j, B_i - w_i)\}$

Proof for (1): $\text{OPT}(i, B_i)$ is the maximum value when item i is in the bag following the constraints. This will always be greater than or equal to any other selection.

Proof for (2) by Induction: *Base Case:* $i = 1$

let us reiterate our recurrence relation as

$$\text{DP}(i, B_i) = c_i + \max_{j < i} \{\text{DP}(j, B_i - w_i)\}$$

$DP(i, B_i)$; is the cost when item 'i' is added to the optimal subset of $[i-1]$ whose cost is at most $B_i - w_i$.

$OPT(i, B_i)$; is the true maximum for the subset of $[i]$ and including item i.

We need to prove

$$DP(i, B_i) \geq OPT(i, B_i)$$

$$\text{RHS : } OPT(1, B_1) = OPT(1, w_1) = c_1$$

The only subset of $\{1\}$ that includes item 1 is $\{1\}$ itself. So the cost is c_1 .

$$\text{LHS : } DP(1, w_1) = c_1 + DP(0, 0) = c_1 + 0 = c_1$$

Thus $DP(1, w_1) = OPT(1, w_1)$; base case holds.

Inductive Hypothesis: Assume that for all $j < i$ we have $DP(j, w') \geq OPT(j, w')$ for all $w' \leq B$.

Now we need to prove $DP(i, B_i) \geq OPT(i, B_i)$.

Let S be the optimal subset of $\{1, 2, \dots, i\}$ that includes item i with total weight $\leq B_i$.

By definition, its cost is $OPT(i, B_i)$.

Consider the set $S' = S \setminus \{i\}$. This is the subset remaining after we remove item i . S' must have a “highest index” item. Let's call that index k . By construction, $0 \leq k < i$.

The weight of $S' = B_i - w_i$. The cost of $S' = OPT(k, B_i - w_i)$.

By our inductive hypothesis:

$$DP(k, B_i - w_i) \geq OPT(k, B_i - w_i)$$

Adding c_i to both sides:

$$c_i + DP(k, B_i - w_i) \geq c_i + OPT(k, B_i - w_i)$$

The LHS is $DP(i, B_i)$ by definition. The RHS is equivalent to adding item i to the optimal set with budget $\leq B_i - w_i$ which becomes $OPT(i, B_i)$. Therefore:

$$DP(i, B_i) \geq OPT(i, B_i)$$

□

Time Complexity: To compute $OPT(i, B_i)$, we take $O(i-1)$ time and number of entries in matrix $= nB$ (n rows and B columns). So, total running time $= O(n^2B)$.

Notice that this is not a polynomial in n . We also have B which is the largest integer involved in defining the problem. We call such algorithms *pseudo-polynomial*.

We could have done better (i.e.) $O(nB)$. The trick is to not decide that item i will be in the optimal set. We will explore this further in the next problem.

Problem 2 (Knapsack Extension). *There is a road and we have to put some signboards at the given locations on the road. Each signboard brings some revenue.*

Constraint: No two signboards are within 5 km of each other.

Given:

- $x_1, x_2, \dots, x_n$ – locations on road
- $r_1, r_2, \dots, r_n$ – revenues at each location

where $x_i \in \mathbb{R}$ and $r_i \in \mathbb{R}$.

Distance between x_i and $x_j = |x_i - x_j|$.

Goal: Maximize total revenue.

Solution. Let $\text{OPT}(i)$ be the total revenue obtained when considering only locations $x_1, x_2, \dots, x_i$ with the constraint that there is a signboard at x_i in the optimal solution.

Then:

$$\text{OPT}(i) = r_i + \max_{\substack{j < i \\ |x_i - x_j| \geq 5}} \text{OPT}(j)$$

and final solution is $\max_{i \in [n]} \{\text{OPT}(i)\}$.

Proof of Correctness: We just need to prove the recurrence relation stated above. We need to prove both the inequalities $\geq$ and $\leq$ like in the previous problem.

Time Complexity: $O(n^2)$ since each $\text{OPT}(i)$ takes $O(i - 1)$ time and we compute for all $i \leq n$.

Can we do Better? Yes!!

Let $\text{OPT}(i)$ be the total revenue obtained considering the given locations are $x_1, x_2, \dots, x_i$ (we don't care if a signboard is at x_i).

Now:

$$\text{OPT}(i) = \max \begin{cases} \text{OPT}(i - 1) \\ r_i + \text{OPT}(j) \text{ such that } x_i - x_j \geq 5 \text{ and } x_i - x_{j+1} < 5. \end{cases}$$

and final solution = $\max_{i \in [n]} \text{OPT}(i)$.

Here, the critical step is to find the best j for each i . We need to find j such that x_j is to the left of x_i and closest to x_i but at a distance of ≥ 5 .

Since x 's are in ascending order, we need not check at all x_j 's from x_1 to x_i . We can just use binary search to find the best x_j .

This way: each $\text{OPT}(i)$ takes $O(\log i)$ time. We have to compute $\forall i \leq n$ So, $O(n \log n)$.

Can we do Better? Yes!! in Just $O(n)$ If we can find the optimal j for all i in just one pass of the x -array, then the time complexity of the algorithm will become just $O(n)$ because we will be storing the optimal j for each i and we can retrieve it in $O(1)$. And then, in a single pass, we will be calculating $\text{OPT}(i)$ starting from $i = 1$ and keep storing the $\text{OPT}(i)$ for each i so that we can use those values afterwards. So, the time complexity becomes just $O(n)$.

Now we have to devise an algorithm to find the optimal j for all i in a single pass.

Algorithm to find optimal j for i : Let the location array be $x = [x_1, x_2, x_3, \dots, x_{n-2}, x_{n-1}, x_n]$.

Let the answer array be $\text{optimal} = [0, 0, 0, \dots, 0, 0, 0]$ (size n).

We start the pointer at x_n and move towards left. Initially, $p = n$ and $i = n$ (considering indexing from 1 to n).

```
while (i > 0 && p > 0) {
    if (x_i - x_p >= 5) {
        optimal[i] = p;
        i--;
    } else {
        p--;
    }
}
```

□

This two-pointer approach finds all optimal j values in $O(n)$ time. □

Problem 3 (Sequence Alignment). *Given the strings X and Y , we have to match the strings X and Y such that the misalignment cost is minimum (for each gap, the cost is δ and for a mismatch of x_i and y_j , the cost is $\text{cost}(x_i, y_j)$).*

Solution. Let $\text{OPT}[i, j]$ be the minimum cost of matching the strings $[x_1, x_2, \dots, x_i]$ and $[y_1, y_2, \dots, y_j]$.

The recurrence relation is:

$$\text{OPT}[i, j] = \min \begin{cases} \text{cost}(x_i, y_j) + \text{OPT}[i-1, j-1] \\ \delta + \text{OPT}[i-1, j] \\ \delta + \text{OPT}[i, j-1] \end{cases}$$

If $x_i = y_j$, then $\text{cost}(x_i, y_j)$ will be 0, then $\text{OPT}[i, j]$ is just $\text{OPT}[i-1, j-1]$.

and the final answer is $\text{OPT}[m, n]$ where $m = \text{length}(X)$ and $n = \text{length}(Y)$.

Time Complexity: We will form a DP matrix like we did for the Knapsack problem.

$$\text{OPT}[0, 0] = 0$$

$\forall i \quad \text{OPT}[i, 0] = i\delta$ because putting i gaps is the only way to match i -letter word and 0-letter word
 $\forall j \quad \text{OPT}[0, j] = j\delta$ because putting j gaps is the only way to match j -letter word and 0-letter word

We will build the matrix row by row. $\text{OPT}(1, 1)$ will use values of $\{\text{OPT}(0, 0), \text{OPT}(1, 0), \text{OPT}(0, 1)\}$
 $\Rightarrow$ So, $O(1)$.

$\text{OPT}(2, 1)$ will use values of $\{\text{OPT}(1, 0), \text{OPT}(2, 0), \text{OPT}(1, 1)\} \Rightarrow$ So, $O(1)$.

This way we need to fill all the remaining $m \times n$ cells of the matrix (first row and first column are filled already).

So, time Complexity = $O(mn)$.

□

Problem 4 (Sequence Alignment Extension). *There is a set of strings L of size k : $L = \{l_1, l_2, \dots, l_k\}$.*

There is a string S of length n . We have to merge the l_i 's to get a string S' (a string l_i can be used more than once). We have to find the minimum cost between S and S' (i.e.) the cost of alignment for the optimal S' .

Solution. Let $\text{len}(S) = n$ and $L = \{l_1, l_2, \dots, l_k\}$.

Assume we have a function $\text{find_cost}(X, Y)$ from the previous problem which gives us the minimum cost of alignment.

Let $\text{DP}(i)$ be the minimum cost to align the string $S[1 \dots i]$ with some concatenation of strings from L .

Then:

$$\text{DP}(i) = \min_{\substack{0 \leq p < i \\ 1 \leq j \leq k}} \{\text{DP}(p) + \text{find_cost}(S[p+1 \dots i], l_j)\}$$

and final answer is $\text{DP}(n)$.

Initialization: Create a DP array of size $n + 1$.

$\text{DP}[0] = 0$ ($\because$ if S is an empty string, we don't have to match anything)

$\text{DP}[i] = \text{Inf}$ for all other i

Iteration:

- For each index i from 1 to n :
 - For each string $l_j \in L$:
 - * For every possible split point p ($0 \leq p < i$):
 - use $\text{find_cost}(S[p+1 \dots i], l_j)$
 - update $\text{DP}(i)$

$\text{DP}[n]$ will give the final answer.

Time Complexity: $O(n^3km)$ Let's say length of a string $l_j = m$.

Each time we are calling the `find_cost()` function, it takes $O(m \cdot (i - p))$ time.

The outermost loop runs for n times.

The second loop runs for k times.

The third loop runs from 1 to i .

So, total running time = $\sum_{i=1}^n \sum_{j=1}^k \sum_{p=0}^i (i - p)m \approx O(n \cdot k \cdot n \cdot nm) = O(n^3km)$. $\square$